FULL-STACK WEB APPLICATION

# Collab Kanban

## Real-Time Collaboration Board
## Complete Project Documentation

Production-ready kanban board with live multi-user sync, optimistic updates aur six major edge-case handling. Next.js 16 + Socket.io + PostgreSQL (Neon) stack, Render par deployed. Architecture, API design, security, deployment aur interview preparation - sab kuch is document mein covered hai.


**Nikhil Sharma**

APPSICA Machine Test - Q3: Real-Time Collaboration Board
Repository: github.com/nikhilsir/collab-taskboard


**Live URL** collab-taskboard.onrender.com

**Stack** Next.js 16 · Socket.io · Prisma · PostgreSQL · Render


OCTOBER 2026 · VERSION 1.0

# Table of Contents

Chapter 1: Project Overview 1

Chapter 2: Technology Stack 2

Chapter 3: System Architecture 3

Chapter 4: Database Design 5

Chapter 5: REST API Design 6

Chapter 6: Real-Time Layer (Socket.io) 7

Chapter 7: Frontend Architecture 9

Chapter 8: Optimistic Updates and Version Conflict Handling 10

Chapter 9: Edge Cases Handling (6 Major) 11

Chapter 10: Security Implementation 13

Chapter 11: Deployment Guide (Neon + Render) 14

Chapter 12: Testing and 2-User Real-Time Verification 15

Chapter 13: Performance and Limitations 16

Chapter 14: Future Improvements 17

Chapter 15: Interview Questions and Answers 18

Chapter 16: Quick Reference Cheat Sheet 20

# Chapter 1: Project Overview

CollabKanban ek real-time collaboration board hai - Trello ya Jira jaisa lightweight tool, jo maine APPSICA machine test ke Q3 round ke liye banaya hai. Requirement simple thi: ek aisa kanban board jo multiple users ke saath same time par kaam kare, matlab ek user jo card drag kare, dusre user ki screen par wahi change turant reflect ho - bina refresh kiye. Isi single requirement ke andar bahut saare tricky engineering problems chhupe hain, aur yahi is project ko interesting banati hai.

Ye ek pure web application hai, sirf ek static website nahi. Ismein user accounts, authentication, database, server-side logic aur real-time communication - sab kuch hai. Simple words mein: frontend React ka hai, backend Node.js ka hai, aur beech mein ek WebSocket layer hai jo sab users ko sync mein rakhti hai. Project fully deployed hai aur live URL se koi bhi access kar sakta hai.

## Core Features

- **Authentication** - Email + password se register/login, JWT token based sessions, bcrypt password hashing.
- **Boards aur Membership** - Har user apne boards bana sakta hai, aur email se dusre users ko invite kar sakta hai.
- **Tasks (Cards)** - Title, description, assignee, due date; columns ke beech drag-and-drop movement.
- **Real-Time Sync** - Koi bhi change (create/update/move/delete) sab members ki screen par instant update.
- **Presence** - Kaun kaun abhi board par online hai, avatars header mein dikhte hain.
- **Typing Indicator** - Task modal mein koi comment type kar raha hai to dusre users ko dikhta hai.
- **Comments aur Activity Log** - Har task par discussion, aur board ka pura history timeline.

## Important Links

| Item | URL / Value |
|---|---|
| Live Application | https://collab-taskboard.onrender.com |
| GitHub Repository | https://github.com/nikhilsir/collab-taskboard (main branch) |
| Database | Neon PostgreSQL - AWS Singapore region |
| Hosting | Render Web Service (free tier) |
| Production commit | d400388 - single-server architecture |

Table 1.1: Project ke key links aur deployment details

Is documentation ka goal ye hai ki koi bhi technical interviewer - ya khud main future mein - is project ko end-to-end samajh sake. Chapters 3 se 10 tak architecture, database, API, real-time layer aur saare engineering decisions detail mein cover kiye gaye hain. Chapter 11 deployment ki step-by-step guide hai, aur

Chapter 15 mein interview questions ke ready answers hain. Ye document banane ke baad confidence level bilul alag hai - kyunki likhte waqt hi har decision dobara verify hota hai.

# Chapter 2: Technology Stack

Tech stack choose karte waqt maine do baatein dhyan rakhi: pehli, stack modern ho aur interview mein relevant ho; dusri, deployment free tier par possible ho. Neeche wali table mein har layer ka technology aur uske peeche ka reason diya gaya hai. Har choice deliberate hai - koi bhi library "kyunki sab use karte hain" wali feeling se add nahi ki.

| Layer | Technology | Kyun Chosen |
|---|---|---|
| Frontend Framework | Next.js 16 + React 19 | App Router, server-side rendering, industry standard, TypeScript built-in support. |
| Language | TypeScript | Compile-time type safety - runtime bugs kam, refactoring easy, interviewer favourite. |
| Styling | Tailwind CSS 4 | Fast development, consistent design system, no CSS naming debates. |
| UI Components | shadcn/ui | Copy-paste accessible components - Radix UI primitives ke upar built. |
| State Management | Zustand | Redux se halka, boilerplate nahi, perfect for board state + optimistic updates. |
| Backend | Node.js 22 + Next.js API Routes | Ek hi language full-stack, single-server deployment possible. |
| Real-Time | Socket.io 4 | Auto-reconnection, rooms, fallback transports - raw WebSocket ka upar abstraction. |
| Database | PostgreSQL (Neon) | Relational data (users/boards/tasks) ke liye natural fit, Neon serverless + free tier. |
| ORM | Prisma | Type-safe queries, schema-first design, migrations aur db push built-in. |
| Auth | JWT (jose) + bcrypt | Stateless auth - server ko session store nahi rakhna padta, horizontal scaling friendly. |
| Hosting | Render | Free web service, GitHub auto-deploy, environment variables UI. |

Table 2.1: Complete technology stack with rationale

## Stack ke Important Decisions

Sabse pehla decision tha **Next.js full-stack rakhna** - alag Express server nahi banaya. Isse ek hi repository, ek hi deployment, aur same-origin API calls mile. Doosra decision tha **Socket.io vs raw WebSocket**: raw WebSocket mein reconnection, room management aur fallback sab khud likhna padta, jabki Socket.io ye sab out-of-the-box deta hai - production mein yahi chahiye hota hai. Teesra decision **PostgreSQL vs MongoDB** ka tha: kanban board ka data heavily relational hai (user-board-task-comment hierarchy, foreign keys, joins), isliye Postgres zyada natural choice hai.

| 6 | 1 | 9/9 | 100% |
|---|---|---|---|
| Major Libraries | Single Server | Tests Passing | TypeScript Backend |

# Chapter 3: System Architecture

Architecture ka sabse bada design decision ye hai: **poora backend ek hi server process mein hai**. Next.js app, API routes aur Socket.io gateway - teeno ek Node.js process (server.mjs) ke andar chalte hain, jo Render par deploy hai. Neeche diye gaye diagram mein poora system dikh raha hai - client layer se lekar database layer tak.

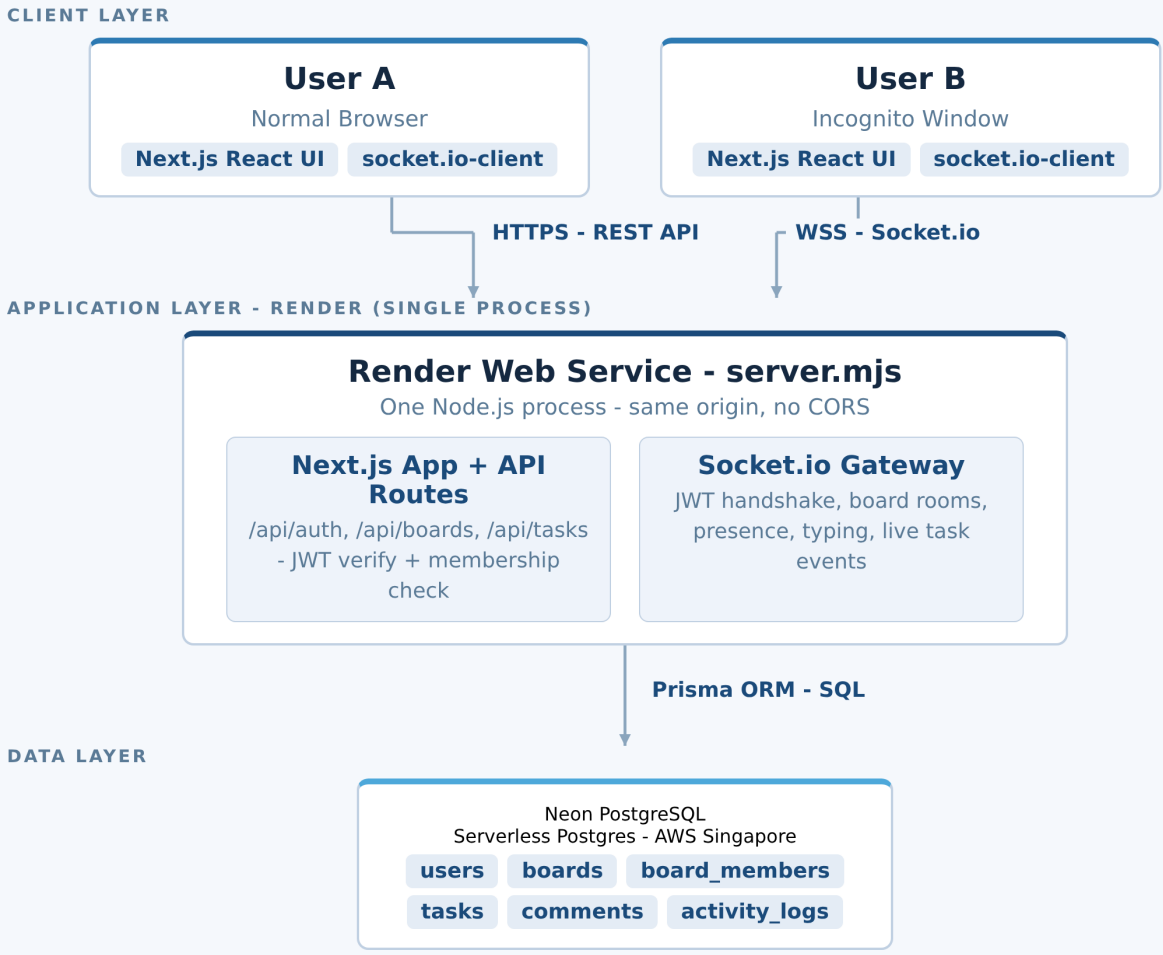



Figure 3.1: CollabKanban single-server architecture - client, application aur data layer

## Single-Server Architecture - Kyun?

Zyada tar tutorials mein Socket.io alag server (port 3001) par chalta hai aur Next.js alag port (3000) par. Is approach mein do problems aati hain: pehli, CORS configuration har jagah chahiye; dusri, Render jaise platforms par do services deploy karni padti hain aur free tier dobara consume hota hai. Maine dono ko ek process mein daal diya - server.mjs Next.js ko programmatic API se createServer ke through chalata hai aur usi HTTP server par Socket.io attach karta hai.

Is design ke concrete fayde ye hain: **no CORS** (sab kuch same origin collab-taskboard.onrender.com se serve hota hai), **ek hi deploy target** (Render service ek hi hai), aur **shared code** (JWT verification, membership check jaise helpers API routes aur socket handlers dono use kar sakte hain). Internal events ke liye ek SOCKET_INTERNAL_SECRET environment variable hai, taaki koi external aadmi internal socket events ko emit na kar sake.

## Data Flow - Ek Task Move Karne Ki Journey

- User A card ko "To Do" se "In Progress" drag karta hai - UI turant optimistic update dikhata hai.
- Frontend POST /api/tasks/[taskId]/move ko JWT ke saath call karta hai (naya column, position, version).

- API route token verify karta hai, board membership check karta hai, version conflict check karta hai.
- Postgres mein task update hota hai (version +1, naya position), ActivityLog mein entry jaati hai.
- API route socket-emit helper se board room mein task:moved event broadcast karta hai (sender ko chhod kar).
- User B ka socket client event receive karta hai, eventId dedup check karta hai, aur UI update ho jaata hai.

# Chapter 4: Database Design

Database PostgreSQL hai aur schema Prisma se manage hota hai (production ke liye prisma/schema.postgres.prisma). Design simple-normalized hai: six tables, clear foreign keys, aur indexes jo query patterns ko support karte hain. Neeche har table ka purpose aur important fields diye hain.

| Table | Important Fields | Purpose |
|---|---|---|
| User | id (cuid), email (unique), passwordHash, name, avatarColor | Registered users; password bcrypt hash ke roop mein store hota hai. |
| Board | id, title, ownerId -< User | Kanban board; owner user ka reference rakhta hai. |
| BoardMember | boardId + userId (composite PK), role | Many-to-many: kaun kis board ka member hai; invite yahin se hota hai. |
| Task | id, boardId, title, description, column, position (decimal), version (int), assigneeId, dueDate | Cards; column + position se ordering; version optimistic locking ke liye. |
| Comment | id, taskId -< Task, authorId -< User, body, createdAt | Task-level discussion thread. |
| ActivityLog | id, boardId, actorId, action, payload (JSON), createdAt | Audit trail - board ka pura history timeline view. |

Table 4.1: Database schema overview (six tables)

## Column Ordering - Fractional Indexing

Cards ke order ko maintain karne ke liye position naam ka **decimal column** hai. Jab naya card beech mein insert hota hai, to uski position = (upar wali card ki position + neeche wali ki position) / 2. Isse kabhi bhi poori column reshuffle nahi karni padti - sirf ek row update hoti hai. Ye approach "fractional indexing" kehlati hai, aur iski detail Chapter 9 mein edge case ke roop mein di hai.

```
Excerpt - prisma/schema.postgres.prisma
model Task {
id String @id @default(cuid())
boardId String
```

```
column String // "todo" | "inprogress" | "done"
position Float // fractional indexing for ordering
version Int @default(1) // optimistic locking
// ... relations
@@index([boardId, column, position])
}
```

Composite index (boardId, column, position) sabse important index hai, kyunki board kholne par wahi query chalti hai - "is board ke saare tasks, column ke hisaab se, position order mein". Iske bina har board load par full table scan hota. Version column default 1 se start hota hai aur har successful update par increment hota hai - yahi optimistic locking ka base hai jo Chapter 8 mein detail se samjhaya hai.

# Chapter 5: REST API Design

API design REST conventions follow karti hai: resource-based URLs, proper HTTP verbs, aur meaningful status codes. Har protected route JWT verify karta hai, aur board-scoped routes membership bhi check karte hain - matlab sirf logged-in hona kaafi nahi hai, us board ka member hona bhi zaroori hai. Neeche complete endpoint table hai.

| Method | Endpoint | Purpose | Auth |
|---|---|---|---|
| POST | /api/auth/register | Naya user create (email, password, name) | Public |
| POST | /api/auth/login | Login, JWT token return | Public |
| GET | /api/auth/me | Current user profile (token se) | JWT |
| GET | /api/boards | Apne sab boards list | JWT |
| POST | /api/boards | Naya board create | JWT |
| GET | /api/boards/[boardId] | Board detail + tasks + members | JWT + Member |
| POST | /api/boards/[boardId]/invite | Email se member invite | JWT + Member |
| POST | /api/tasks | Naya task create (position auto-calc) | JWT + Member |
| PATCH | /api/tasks/[taskId] | Task update (title, desc, assignee, version) | JWT + Member |
| POST | /api/tasks/[taskId]/move | Task ko column+position move (version check) | JWT + Member |
| POST | /api/tasks/[taskId]/comments | Task par comment add | JWT + Member |

| Method | Endpoint | Purpose | Auth |
|---|---|---|---|
| GET | /api/users/search | Invite ke liye email se user search | JWT |
| POST | /api/internal/check-membership | Socket handshake ke liye membership verify | Internal Secret |

Table 5.1: Complete REST API surface (13 endpoints)

## Move Endpoint - Request / Response Example

```
Example - move request with version check
POST /api/tasks/ckt9x.../move
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
{ "column": "inprogress", "position": 150.5, "version": 3 }

200 OK -> { "task": { ..."version": 4 } }
409 Conflict -> { "error": "VERSION_CONFLICT", "tasks": [fresh rows] }
403 Forbidden -> membership check fail
```

Is API ka sabse interesting response **409 Conflict** hai. Jab client jo version bhejta hai wo database ke current version se match nahi karta, matlab beech mein kisi aur ne task badal diya - server 409 deta hai aur saath mein **fresh task rows** return karta hai. Client ko dobara fetch karne ki zaroorat nahi padti, response body mein hi resync ka data mil jaata hai. Ye design decision Chapter 8 mein poora explain kiya hai.

Ek alag endpoint notice karo: /api/internal/check-membership. Ye public nahi hai - isko sirf server ke andar Socket.io handshake waqt call kiya jaata hai, isliye iska auth normal JWT nahi balki SOCKET_INTERNAL_SECRET hai. Isse guarantee hoti hai ki koi bhi user us board room se connect na ho jaye jiska wo member hi nahi hai.

# Chapter 6: Real-Time Layer (Socket.io)

Real-time communication ka poora load Socket.io handle karta hai, jo same Node.js process mein Next.js ke saath chalta hai. Har connection pehle authenticate hota hai, phir specific board rooms mein join hota hai, aur uske baad hi events send/receive kar sakta hai. Ye three-step security model - handshake auth, room membership, event validation - Chapter 10 mein security lens se bhi cover kiya hai.

## Connection Lifecycle

- **Step 1 - JWT Handshake:** Client connect ke waqt token bhejta hai (auth payload). Server jose.verify se token validate karta hai; invalid token par connection turant reject.
- **Step 2 - Room Join:** Client board:boardId room join karta hai. Server us user ki membership re-check karta hai (DB query) - sirf members ko join allowed.

- **Step 3 - Live Events:** Room ke andar sab events broadcast hote hain; server har event par sender ko chhod kar baaki members ko bhejta hai.
- **Step 4 - Disconnect:** Disconnect par presence update sabko jaata hai aur cleanup hota hai (Chapter 9, Edge Case 5).

## Socket Events Reference

| Event | Direction | Payload (main fields) | Kaam |
|---|---|---|---|
| task:created | Server to Clients | task, eventId, actor | Naya card sabki screen par add. |
| task:updated | Server to Clients | task, eventId, actor | Title/desc/assignee changes. |
| task:moved | Server to Clients | taskId, column, position, version, eventId | Drag-drop sync. |
| task:deleted | Server to Clients | taskId, eventId, actor | Card removal. |
| presence:update | Server to Clients | onlineUsers[] (id, name, color) | Online avatars header mein. |
| typing:start / stop | Client to Server | taskId, user | Comment box typing indicator. |
| activity:new | Server to Clients | activity entry | Activity panel live feed. |

Table 6.1: Socket.io event reference

## EventId - Dedup System

Har server event mein ek unique **eventId** hota hai. Client har received eventId ko ek Set mein yaad rakhta hai aur duplicate event ko silently drop kar deta hai. Ye zaroori hai kyunki Socket.io reconnection ke baad "at-least-once" delivery jaisa behaviour dikha sakta hai - same event do baar aana possible hai, aur bina dedup ke UI mein duplicate cards ya flickering dikhega. Ye chhota sa mechanism poore system ki idempotency ka backbone hai.

Ek aur subtle point: server events broadcast karte waqt **sender ko exclude** karta hai, kyunki sender ne pehle hi optimistic update se UI change kar liya hota hai. Agar sender ko bhi event milta, to dobara apply hone par janky behaviour hota (card dobara jump karna). Socket broadcast mein room.emit ki jagah socket.to(room).emit use hone ka yahi reason hai.

# Chapter 7: Frontend Architecture

Frontend Next.js App Router par bana hai aur components do categories mein divided hain: page-level screens (auth, board list, board view) aur kanban components (board, columns, cards, modals). State ka single source of truth Zustand store hai, aur real-time layer ek custom hook ke through components se juda hai. Is separation ki wajah se socket logic UI se independent testable rehta hai.

| Component / Module | Responsibility |
|---|---|
| auth-screen.tsx | Register/login forms, JWT ko localStorage mein save. |
| board-header.tsx | Board title, online members avatars (presence), invite button. |
| board-sidebar.tsx | Board list navigation, current user info, logout. |
| kanban-board.tsx | Board container - columns render karta hai, socket events subscribe. |
| column.tsx | Ek column ke cards, drop target logic, position calculate karna. |
| task-card.tsx | Single card UI - drag source, title, labels, assignee avatar. |
| task-modal.tsx | Task detail edit + comments + typing indicator. |
| invite-dialog.tsx | Email search karke member invite karna. |
| activity-panel.tsx | Board ka live activity timeline. |
| store.ts (Zustand) | Tasks, columns, presence, optimistic update queue - single store. |
| kanban-socket.ts (hook) | Socket connect, room join, event listeners, cleanup - poora realtime lifecycle. |

Table 7.1: Frontend module map

## State Management - Zustand Kaise Use Kiya

Board ka poora state (tasks, column order, online users, typing users) ek Zustand store mein rehta hai. Optimistic updates isi store ke through hote hain: jab user card drag karta hai, pehle store update hota hai (UI turant respond karta hai), phir API call jaati hai. API fail ho to store ko previous state par rollback karte hain - ye pattern Chapter 8 ka core hai. Zustand yahan isliye best hai kyunki selector-based subscriptions se sirf relevant components re-render hote hain, poora board nahi.

Socket listeners ko components ke andar direct use karne ki jagah maine ek custom hook (kanban-socket.ts) banaya. Ye hook mount par connect + join karta hai, sab event listeners attach karta hai, aur unmount par poora cleanup - listeners remove, socket disconnect, timers clear. Is pattern se "socket cleanup" edge case (Chapter 9) systematically solve hota hai aur koi memory leak nahi hota.

# Chapter 8: Optimistic Updates and Version Conflict Handling

Ye chapter is project ka technical heart hai - interview mein sabse zyada sawaal isi par aate hain. Do concepts saath kaam karte hain: **optimistic updates** (UI responsiveness ke liye) aur **version-based optimistic locking** (data consistency ke liye). Dono ka combination hi multi-user boards ko race conditions se bachata hai.

## Optimistic Update - Kya aur Kyun

Optimistic update ka matlab: server ka response **wait kiye bina** UI ko update kar dena, kyunki hum "optimistically" maan lete hain ki request succeed hogi. User card drag karta hai - card turant naye column mein chala jaata hai; background mein API call jaati hai. Server 200 deta hai to kuch nahi karna (UI already sahi hai); server fail deta hai to UI rollback - card wapas original column mein, saath mein user ko halka error toast. Isse interaction latency feel hoti hai ~0ms, chahe network 300ms le ho.

```
Optimistic move + rollback flow
// kanban-board.tsx (simplified)
const prev = useStore.getState().snapshot();
store.moveLocal(taskId, column, position); // 1. optimistic UI
const res = await api.moveTask(taskId, {..., version});
if (!res.ok) {
store.restore(prev); // 3. rollback
if (res.status === 409) store.resync(res.body.tasks); // 4. fresh data
toast.error('Move failed - board refreshed');
}
```

## Version Field - Optimistic Locking

Ab soch: User A aur User B ek hi card ko same second mein move kar dete hain. Dono ke paas card ka same version tha (maan lo 3). A ka request pehle pahunchti hai - server version 3 dekh ke update karta hai, version 4 ho gaya. B ki request pahunchti hai version 3 ke saath - server check karta hai: DB version 4 hai, client 3 bhejta hai - **conflict!** Server 409 return karta hai fresh task data ke saath, aur B ka client UI resync kar deta hai. Isi mechanism ko optimistic locking kehte hain - database par lock lagaye bina consistency.

| Time | User A (Browser 1) | User B (Browser 2) | Server / DB |
|---|---|---|---|
| t1 | Card version 3 load hua | Card version 3 load hua | version = 3 |
| t2 | Card ko drag kiya, UI turant moved | Kuch nahi kiya | - |
| t3 | Move request: version 3 bheja | - | Update OK, version = 4 |

| Time | User A (Browser 1) | User B (Browser 2) | Server / DB |
|---|---|---|---|
| t4 | 200 OK - koi change nahi | Card drag kiya, UI moved | - |
| t5 | - | Move request: version 3 bheja | DB version 4 hai - 409 CONFLICT |
| t6 | - | Response ke fresh tasks se resync | version = 4 confirmed |

Table 8.1: Two-user version conflict timeline

### Kyun Ye Combo Zaroori Hai

Agar sirf optimistic update ho aur version check nahi, to last-write-wins ho jaata - B ka move A ke move ko chupchaap overwrite kar deta, aur A ko pata bhi nahi chalta. Agar sirf version check ho aur optimistic update nahi, to har drag ke baad response ka wait karna padta - board sluggish lagta. Dono ka combo hi production-grade experience deta hai: **fast UI + guaranteed consistency**. Trello/Jira jaise products bhi isi family ke patterns use karte hain.

> *Interview one-liner: "Maine optimistic locking implement ki hai - har task mein ek version field hai, client stale version se update karne ki koshish kare to server 409 deta hai fresh data ke saath, aur client apne optimistic UI ko resync kar leta hai."*

# Chapter 9: Edge Cases Handling (6 Major)

Machine test mein sirf "kaam karne wala" app kaafi nahi hota - edge cases hi ordinary aur production-grade work mein difference daalte hain. Maine apne project mein six major edge cases identify kiye aur har ek ka explicit solution implement kiya. Is chapter mein har edge case ka format same hai: **Problem - Solution - Kaise test kiya**. Interview mein ye chapter sabse zyada kaam aayega.

## Edge Case 1: Version Conflict (409 to Resync)

**Problem:** Do users same card ko almost same time par move/update kar dein to last-write-wins silently ek user ka change overwrite kar deta. **Solution:** Har task mein version integer hai; update request mein client version bhejta hai, server mismatch par 409 + fresh rows return karta hai, client resync kar leta hai (poora timeline Chapter 8 ke Table 8.1 mein hai). **Test:** Do browsers mein same card pehle move kiya dono ne rapid sequence mein - dusre browser mein board auto-correct hua, koi stale card nahi bacha.

## Edge Case 2: Fractional Indexing (Card Ordering)

**Problem:** Agar position integer hoti, to beech mein card insert karne ke liye poore column ki positions reshuffle karni padti - N database updates. **Solution:** position decimal hai; naya card position = (prev + next) / 2. Sirf ek row update. **Test:** 5 cards ke beech repeated insertions kareen - order hamesha correct, DB mein

position values 1000, 750, 625... pattern mein decaying, koi collision nahi. Extreme cases ke liye re-indexing function bhi rakha hai (jab gap too small ho jaye).

## Edge Case 3: EventId Dedup (Duplicate Socket Events)

**Problem:** Socket.io reconnect hone par same event duplicate deliver ho sakta hai - bina dedup ke UI mein card duplicate ya double-animation dikhega. **Solution:** Server har event ko unique eventId deta hai; client Set maintain karta hai, already-seen event drop. **Test:** Network throttle karke reconnect trigger kiya - tasks exactly ek baar hi apply hue.

## Edge Case 4: Optimistic Update Rollback

**Problem:** Optimistic UI fail ho to? Card to move ho gaya tha screen par, par server ne reject kar diya (network down / 409 / permission). **Solution:** Request se pehle store ka snapshot lete hain; failure par snapshot restore + 409 par fresh resync + error toast. **Test:** DevTools se network offline karke drag kiya - card wapas original column mein aa gaya, toast dikha, board state consistent raha.

## Edge Case 5: Socket Cleanup (Memory Leaks)

**Problem:** Board page se navigate karne par listeners remove nahi hue to ghost listeners accumulate hote hain - duplicate events, memory leak, aur "phantom updates" (jo page khol chuka tha wo ab bhi events apply kar raha hai). **Solution:** Sab socket lifecycle ek custom hook mein centralized; useEffect return mein listeners off + socket disconnect + typing timers clear. **Test:** Board se navigate karke wapas aane ko 10 baar repeat kiya - har baar ek hi connection bana, DevTools ke network tab mein socket count constant raha.

## Edge Case 6: JWT Handshake + Board Membership Double Validation

**Problem:** Socket par sirf JWT verify karna kaafi nahi hai - valid token wala koi bhi user kisi bhi board ka room join kar sakta hai aur us board ka live data sun sakta hai (data leak). **Solution:** Do-layer check: handshake par JWT verify, room join par DB se membership verify (via internal check-membership endpoint jo internal secret se protected hai). **Test:** Ek user dusre board ka room join karne ki koshish ki (custom socket client se) - server ne join reject kar diya.

| 6 | 409 | 2 | 0 |
|---|---|---|---|
| Edge Cases Solved | Conflict Handled | Validation Layers | Known Leaks |

# Chapter 10: Security Implementation

Security is project mein afterthought nahi hai - design ke shuru se hi layered security model follow kiya gaya hai. Har request par identity verify hoti hai, har board-scoped operation par membership verify hoti hai, aur passwords kabhi plain text mein store nahi hote. Neeche saare mechanisms detail mein hain.

## Authentication Flow

- **Register:** Email/password receive - password ko bcrypt se hash karke store (10 salt rounds); plain password kabhi log/store nahi.
- **Login:** Email se user fetch, bcrypt.compare se password match, success par JWT sign (jose library, HS256, 7-day expiry).
- **Har Request:** Authorization: Bearer header se token, server-side middleware verify karta hai - payload se userId nikaal kar request context mein daalta hai.
- **Socket Handshake:** Same JWT handshake auth payload mein jaata hai; invalid token par connection reject before any event.

## Authorization - Membership Check

Authentication batati hai **kaun** hai; authorization batati hai **kya kar sakta** hai. Is project mein har board-scoped route (board detail, tasks, invite, comments) par BoardMember table se check hota hai ki requesting user us board ka member hai ya nahi. Ye check API routes aur socket room join - dono jagah hota hai. Isse IDOR (Insecure Direct Object Reference) attacks block hote hain: ek member dusre ke board ka ID guess karke bhi kuch nahi kharoch sakta.

## Environment Secrets

| Variable | Role | Exposure |
|---|---|---|
| DATABASE_URL | Neon Postgres connection string | Server-only |
| JWT_SECRET | Token sign/verify key (HS256) | Server-only |
| SOCKET_INTERNAL_SECRET | Internal membership-check endpoint protection | Server-only |
| NEXT_PUBLIC_SOCKET_URL | Browser ko socket server ka URL | Public (by design) |

Table 10.1: Environment variables aur unki exposure

Ek important security principle jo maine follow ki: **secrets sirf server par**. JWT_SECRET aur DATABASE_URL kabhi NEXT_PUBLIC_ prefix wale variables mein nahi gaye - warna wo browser bundle mein leak ho jaate. Git history mein bhi koi .env file commit nahi hui; .gitignore mein saare env files

excluded hain. Production secrets sirf Render ke environment variable UI mein rehte hain.

# Chapter 11: Deployment Guide (Neon + Render)

Deployment bhi is machine test ka ek graded part hai - "code laptop par chal raha hai" aur "app live URL par hai" mein bada difference hota hai. Maine free-tier stack choose kiya: **Neon** serverless Postgres + **Render** web service. Is chapter mein poora step-by-step process hai, saath mein un pitfalls ke saath jinhe maine khud face kiya, taaki dobara deploy karna ho to 20 minute mein ho jaye.

## Step 1: GitHub Push

- GitHub par empty repo banaya (collab-taskboard), phir git remote add + push -u origin main.
- Fine-grained Personal Access Token banaya - yahan ek pitfall tha: default mein Contents permission **read-only** hota hai, push 403 de deta hai; permission **Read and write** set karna zaroori hai.
- Push se pehle .gitignore verify kiya - .env, node_modules, .next sab excluded; repo mein koi secret leak nahi.

## Step 2: Neon Postgres Setup

- Neon par naya project "collabkanban" banaya - region AWS Singapore (Render se closest).
- Dashboard se connection string copy kiya (pooled nahi - direct host use kiya Prisma ke liye).
- **Pitfall:** Maine pehli baar poori line paste kar di thi - DATABASE_URL="postgresql://..." - variable name aur quotes ke saath. Prisma ne error diya: P1012: URL must start with postgresql://. Fix: sirf pure URL value paste karo, na quotes, na key prefix.

## Step 3: Render Web Service

| Setting | Value |
|---|---|
| Service name | collab-taskboard |
| Runtime | Node (NODE_VERSION=22) |
| Build command | npm install; npx prisma generate --schema prisma/schema.postgres.prisma; npx prisma db push --schema prisma/schema.postgres.prisma; npm run build |
| Start command | npm run start (node server.mjs) |
| Env vars | DATABASE_URL, JWT_SECRET, SOCKET_INTERNAL_SECRET, NEXT_PUBLIC_SOCKET_URL=https://collab-taskboard.onrender.com |

Table 11.1: Render service configuration

Build command mein prisma generate (client banane ke liye) aur prisma db push (schema ko DB mein apply karne ke liye) dono isliye hain, kyunki Render har deploy par fresh environment banata hai. NEXT_PUBLIC_SOCKET_URL ko server URL hi rakha - kyunki socket bhi usi server par hai (single-server architecture ka fayda). Deploy ke baad Render ka build log verify kiya: prisma push success, next build success, server started - aur Live status aa gaya.

### Cold Start - Free Tier Reality

Render free service 15 minute idle ke baad spin down ho jaata hai. Pehli request par instance wapas rehnda hota hai - isliye cold start par 30-50 second lag sakte hain. Ye bug nahi, free tier ki documented behaviour hai. Interview mein batao: "Production mein hum isko paid instance ya keep-alive ping se handle karte, par free tier mein ye expected trade-off hai." Warm hone ke baad response times normal (100-300ms) ho jaate hain.

## Chapter 12: Testing and 2-User Real-Time Verification

Testing do layers par ki hai: automated integration tests (jo poora API flow exercise karte hain) aur manual 2-user real-time verification (jo real browsers mein sab kuch validate karta hai). Dono ka apna role hai - automated tests regression pakadte hain, manual test real UX confirm karta hai.

### Automated Integration Tests - 9/9 Passing

| # | Test | Verify Karta Hai |
|---|---|---|
| 1 | Register new user | 201 + hashed password stored |
| 2 | Duplicate register fails | 409 duplicate email pe |
| 3 | Login correct credentials | 200 + valid JWT |
| 4 | Login wrong password | 401 invalid credentials |
| 5 | Create board | Board row + owner membership |
| 6 | Invite second user | BoardMember row create |
| 7 | Create task | Position auto-assign + version 1 |
| 8 | Move task | Column/position/version update + socket emit |
| 9 | Move with stale version | 409 + fresh tasks payload |

Table 12.1: Integration test suite - sab 9 pass

## Manual 2-User Test Procedure

Real-time behaviour ko test karne ke liye ek systematic procedure follow karo - same browser ke do tabs kaam nahi karenge (session share hota hai). Ek normal window aur ek incognito window lo, alag-alag email accounts se register karo, ek board banao aur dusre user ko invite karo. Uske baad neeche wala checklist run karo.

| Test | Steps | Expected Result |
|---|---|---|
| Drag sync | User A card drag kare | User B ki screen par turant move, no refresh |
| Create sync | User B naya card banaye | User A ko instant dikhe |
| Presence | Dono board par hon | Header mein dono avatars |
| Typing | User A comment type kare | User B ko typing indicator dikhe |
| Conflict | Dono same card rapid move karein | Ek ko 409 resync - board consistent |
| Comment sync | User A comment post kare | User B ko comment + activity dikhe |
| Disconnect | User A tab band kare | User B ke avatars se A hat jaye |

Table 12.2: 2-user manual verification checklist

Ye manual checklist isliye valuable hai kyunki ye wahi scenarios hain jo interviewer live demo ke waqt test karta hai. Main suggest karunga ki demo se pehle ye checklist ek baar zaroor run karo - Render cold start ke baad pehli request slow hoti hai, isliye app ko pehle se warm kar lo (URL ek baar khol do, 30-40 second wait).

# Chapter 13: Performance and Limitations

Har engineering decision trade-off leta hai, aur ek honest documentation mein limitations bhi likhi jaati hain. Ye chapter un constraints ka honest picture deta hai jo free-tier deployment aur single-server design ke saath aate hain - aur har ek ke liye production-grade solution bhi batata hai. Interview mein limitations discuss karne se impression behtar hota hai, kyunki isse dikhta hai ki aap system ki boundaries samajhte ho.

## Known Constraints

| Constraint | Reason | Production Fix |
|---|---|---|
| Cold start 30-50s (idle ke baad) | Render free tier 15-min spin-down | Paid instance / cron keep-alive ping |
| Single instance scaling limit | Socket.io in-memory rooms | Redis adapter + sticky sessions |
| DB connection per request | Neon serverless model | Prisma PgBouncer pooled URL |

| Constraint | Reason | Production Fix |
|---|---|---|
| No file/image upload | Scope control (test focus) | S3/R2 signed URLs |
| Board data unbounded activity log | No retention policy yet | Partitioning / archive job |

Table 13.1: Constraints, reasons aur production fixes

### Performance Jo Achha Hai

- Board load: single indexed query (boardId, column, position) - saare tasks ek hi round-trip mein.
- Socket events: sirf affected task ka payload jaata hai, poora board nahi - bandwidth efficient.
- Optimistic UI: user-perceived latency ~0ms drag/create/update par.
- Zustand selectors: sirf changed-data wale components re-render hote hain.

Scaling roadmap seedha hai kyunki architecture pehle se hi single-process constraints maan kar bana hai: sabse pehle Socket.io ke aage Redis adapter daalenge (multi-instance broadcast), phir Render ko 2+ instances par scale karenge sticky sessions ke saath, aur Neon ka pooled connection string DB layer ke liye. App code mein itni changes nahi chahiye - yahi is design ki asli strength hai.

## Chapter 14: Future Improvements

Is project mein maine consciously scope control kiya - machine test ke core requirements + engineering depth pe focus kiya, aur "nice to have" features ko roadmap mein daala. Ye roadmap realistic hai: har item ka effort estimate aur priority likhi hai, taki koi bhi is project ko aage le jaaye to clear path mile.

| Improvement | Priority | Effort | Notes |
|---|---|---|---|
| dnd-kit drag animations | High | 2-3 din | Current drag functional hai; animation polish UX upgrade dega. |
| Email invite links | High | 1-2 din | Abhi email-search se invite hota hai; magic-link flow better. |
| Notification system | Medium | 3-4 din | Assignment/comments par in-app + email notifications. |
| Board templates | Medium | 2 din | Sprint board, bug tracker jaise presets. |
| Mobile responsive polish | Medium | 2-3 din | Touch drag behaviour + responsive columns. |
| CI/CD (GitHub Actions) | High | 1 din | Push par lint + type-check + tests, deploy gate. |
| Monitoring (Sentry + logs) | Medium | 1-2 din | Error tracking, socket event metrics. |
| Dark mode | Low | 1 din | shadcn/ui already supports - sirf theme toggle. |

Table 14.1: Roadmap with priority and effort

Agar sirf ek cheez implement karni ho to main **CI/CD pipeline** choose karunga - kyunki wo har future improvement ki quality guard karega. GitHub Actions par ek simple workflow: install, type-check, lint, integration tests; pass hone par hi Render auto-deploy. Ye 1 din ka kaam hai par engineering maturity ka strong signal hai.

# Chapter 15: Interview Questions and Answers

Ye chapter is documentation ka sabse important practical section hai - yahan wo sawaal hain jo is project ke interview round mein sabse zyada poochhe jaate hain, har ek ka answer Hinglish mein ready-to-speak format mein. Har answer 30-60 second ka hai - direct, technical, aur ek concrete detail ke saath. Inhe ratne ki nahi, samajhne ki zaroorat hai; concept clear ho to words khud aa jaate hain.

### Q1. Optimistic update kya hai aur aapne kyun use kiya?

"Optimistic update matlab server ka jawab wait kiye bina UI ko update kar dena. Maine ise isliye use kiya kyunki kanban board mein drag-drop ka experience instant hona chahiye - agar har move 300ms ka network wait kare to board sluggish lagta. Main request se pehle Zustand store ka snapshot leta hoon, UI turant update karta hoon, aur agar request fail ho to snapshot restore kar deta hoon - yaani rollback. Isse user ko zero-latency feel hoti hai aur data consistency bhi bani rehti hai."

### Q2. 409 conflict exactly kaise handle hota hai?

"Har task mein ek version integer field hai. Client har update/move request ke saath apna last-seen version bhejta hai. Server check karta hai ki DB ka current version match karta hai ya nahi. Mismatch hone par server 409 Conflict return karta hai, aur response body mein fresh task rows bhi deta hai - taaki client ko dobara GET call na karna pade. Client 409 dekhte hi apne optimistic state ko in fresh rows se resync kar deta hai aur user ko halka toast dikhata hai. Isse last-write-wins problem completely solve ho jaati hai."

### Q3. Fractional indexing kya hai - card ordering kaise handle kiya?

"Har card ka ek position float hai. Column mein naya card aksar beech mein girta hai - agar position integer hoti to poore column ki positions reshuffle karni padti, yani N-1 database updates. Fractional indexing mein naya card ki position = upar wali card aur neeche wali card ki position ka average hota hai - sirf ek row update. Example: 1000 aur 2000 ke beech insert? Position 1500. Repeated insertion par values decay karti hain (750, 625...), aur ek threshold ke baad re-index function poore column ko dobara space de deta hai."

### Q4. Socket.io par authentication kaise kiya - raw connection kaise block kiya?

"Do layers hain. Pehli: Socket.io handshake ke auth payload mein JWT jaata hai, server jose se verify karta hai - invalid token par connection establish hone se pehle hi reject. Dusri: valid token wala user bhi sirf us board ka room join kar sakta hai jiska wo member hai - server room join par database se membership check karta hai ek internal endpoint ke through jo SOCKET_INTERNAL_SECRET se protected hai. Isse data leak

block hota hai ki koi bhi valid user kisi bhi board ka live data sun sake."

### Q5. Duplicate socket events se kaise bacha?

"Har server event mein unique eventId hota hai. Client ek Set maintain karta hai jismein received eventIds hote hain - koi event dobara aaye to wo silently drop ho jaata hai. Ye zaroori hai kyunki Socket.io reconnection ke baad at-least-once delivery behave kar sakta hai. Bina dedup ke UI mein duplicate cards ya double animations dikhte. Idempotency ka ye simple pattern production systems mein bhi same rehta hai."

### Q6. Single-server architecture kyun chuna - alag socket server kyun nahi banaya?

"Teen reasons. Pehla: same origin - CORS ka poora issue khatam, cookies/headers ka complexity nahi. Doosra: deployment simplicity - Render par ek hi service, ek hi build, free tier bhi kaafi. Teesra: code sharing - JWT verify aur membership check jaise helpers API routes aur socket handlers dono use karte hain. server.mjs Next.js ko programmatically createServer se chalata hai aur usi HTTP server par Socket.io attach karta hai. Scaling ka seedha path bhi hai: Redis adapter daal ke multi-instance ho sakta hai."

### Q7. Prisma schema design explain karo - relations kaise hain?

"Six tables hain - User, Board, BoardMember, Task, Comment, ActivityLog. User aur Board ka relation many-to-many hai jo BoardMember join table se hota hai (composite primary key boardId + userId). Task Board se belongs-to hai, aur Comment Task se. Task mein teen special fields hain: column (enum-like string), position (float, fractional indexing ke liye), aur version (int, optimistic locking ke liye). Sabse important composite index (boardId, column, position) hai - board load ki main query isi par chalti hai."

### Q8. Deployment mein sabse bada challenge kya tha?

"Do concrete challenges. Pehla GitHub push: fine-grained token default mein Contents ko read-only rakhta hai, isliye push 403 de raha tha - permission Read and write set karne ke baad push hua. Doosra Neon connection string: maine galti se poori .env line paste kar di thi - quotes aur variable name ke saath - Prisma ne P1012 error diya URL must start with postgresql://. Sirf pure URL value use karne se solve hua. In dono ka learning: error message dhyan se padho - dono baar root cause turant samajh aaya."

### Q9. Cold start kya hai - aapke app ko kholne mein 30 second kyun lagte hain?

"Render ka free tier 15 minute idle ke baad instance spin down kar deta hai. Pehli request par instance wapas boot hota hai - isliye 30-50 second lagte hain, aur ye documented behaviour hai, bug nahi. Warm hone ke baad responses 100-300ms mein hain. Production mein main paid instance ya cron keep-alive ping use karta. Interview mein main ye bhi batata hoon ki maine isko UX mein handle kiya bhi hai - loading state meaningful hai aur user ko context dikhata hai."

### Q10. Agar 1000 users ek board par ho jayein to kya hoga?

"Abhi Socket.io rooms in-memory hain - single instance ke andar scalable hain, par horizontal scaling ke liye Redis adapter chahiye hoga, taaki multiple instances ke beech events share ho sakein. DB ke liye Neon ka pooled connection string (PgBouncer) lagaunga. Presence aur typing events 1000 users par noisy honge - unhe debounce/throttle karna padega ya presence ko sirf active zone tak limit karna padega. Ye classic scale-out path hai aur mera architecture iske liye ready hai."

### Q11. Testing approach kya thi?

"Do layers. Automated: 9 integration tests jo poora API flow cover karte hain - auth (register/login/duplicate/wrong-password), board create + invite, task create, move, aur stale-version 409 case. Ye regression ke liye hain. Manual: 7-step 2-user checklist real browsers mein - drag sync, presence, typing, conflict, disconnect. Real-time behaviour ka confidence manual se hi aata hai kyunki automated socket tests complex hote hain aur real UX pakad nahi paate."

### Q12. Is project mein sabse proud kaunsi cheez ho?

"Honest answer: edge-case handling. Six major edge cases - version conflict resync, fractional indexing, eventId dedup, optimistic rollback, socket cleanup, aur double validation - ye saare cheezein typical tutorial projects mein nahi hoti. Maine ye tabhi soch pa jab maine khud 2-browser testing ki aur socha ki do log ek saath kya-kya galat kar sakte hain. Wo mindset shift - apne code ko todne ki koshish karna - is project ki sabse badi achievement hai."

# Chapter 16: Quick Reference Cheat Sheet

Ye last chapter ek single-page revision sheet hai - demo ya interview se pehle 5 minute mein poora project refresh karne ke liye. Har line compressed hai, aur detail ke liye respective chapter ka reference diya gaya hai. Print karke ya phone par rakh lo - live demo ke waqt yahi sabse kaam aayega.

## Key Facts

| Item | Value |
|---|---|
| Live URL | https://collab-taskboard.onrender.com |
| GitHub | https://github.com/nikhilsir/collab-taskboard |
| Stack one-liner | Next.js 16 + TypeScript + Tailwind + Zustand + Socket.io + Prisma + PostgreSQL (Neon) + Render |
| Architecture | Single Node.js process (server.mjs) - Next.js + API + Socket.io same origin |
| Auth model | JWT (jose, HS256, 7-day) + bcrypt (10 rounds) + membership checks |
| Database | 6 tables: User, Board, BoardMember, Task, Comment, ActivityLog |
| Tests | 9/9 integration tests passing |
| Cold start | 30-50s first request (Render free tier spin-down, expected) |

Table 16.1: Project facts at a glance

## 6 Edge Cases - One Liners

| # | Edge Case | Solution One-Liner |
|---|---|---|
| 1 | Version conflict | version field + 409 + fresh rows resync (Ch 8) |
| 2 | Card ordering | Fractional indexing: position = (prev + next) / 2 (Ch 9) |
| 3 | Duplicate events | eventId Set dedup on client (Ch 9) |
| 4 | Failed optimistic update | Snapshot + rollback + toast (Ch 9) |
| 5 | Socket cleanup | Central hook: unmount par off + disconnect (Ch 9) |
| 6 | Room security | JWT handshake + DB membership check both (Ch 9) |

Table 16.2: Edge cases quick recall

## 30-Second Elevator Pitch

> *Maine ek real-time collaboration kanban board banaya - Next.js 16 aur Socket.io par, PostgreSQL (Neon) ke saath, Render par deployed. Iska sabse strong part engineering depth hai: maine optimistic updates with version-based locking implement kiye, fractional indexing se card ordering solve ki, socket events ko eventId dedup se idempotent banaya, aur socket layer par double validation - JWT handshake plus board membership check - rakha. Nine integration tests passing hain, aur app do real users ke saath live sync hota hai - drag, typing, presence, sab kuch instant.*

Is document ke saath aapka preparation complete hai. Chapters 8 aur 9 ko demo se ek raat pehle dobara padh lena, aur Chapter 12 ka manual checklist live demo se pehle run kar lena. All the best!